

ZC Reclaim: The Digital Lost & Found Hub

Introduction to Software Engineering SW 203

Phase 3: Design Document

Team Members:

- **Mostafa Mohamed Helmy** (202401133)
- **Mohamed Ahmed Ezzat** (202401317)

Representative Contact Info:

s-mostafa.harhash@zewailcity.edu.eg

Date: November 22, 2025

Table Of contents

Introduction

- 1.1 Purpose of the Document
- 1.2 Scope of the Design Phase
- 1.3 Intended Audience
- 1.4 Overview of the Contents

System Overview

- 2.1 Brief Description of the System
- 2.2 Key Design Goals and Constraints

Architectural Design

- 3.1 System Architecture Diagram
- 3.2 Discussion of Architectural Style and Components
- 3.3 Technology Stack and Tools

Detailed Design

- 4.1 Model-View-Controller (MVC) Design Pattern
 - 4.1.1 Description of MVC Pattern
 - 4.1.2 Mapping of Project Components to MVC
 - 4.1.3 Responsibilities
 - 4.1.4 Interaction Flow
- 4.2 UML Diagrams
 - 4.2.2 Detailed Class Diagram
 - 4.2.3 Sequence Diagrams
- 4.3 UI/UX Design
 - 4.3.1 Wireframes / Mockups
- 4.4 Data Design
 - 4.4.1 Database Schema
 - 4.4.3 Data Dictionary

Conclusion

- 5.1 Summary of Design Phase

1. Introduction

1.1 Purpose of the Document

The purpose of this Design Document is to provide a comprehensive architectural and detailed design for **ZC Reclaim**, the centralized digital lost and found hub for Zewail City University. This document translates the functional and non-functional requirements defined in the Phase 2 SRS into technical specifications. It serves as the primary blueprint for the implementation phase, ensuring the system meets all stakeholder needs.

1.2 Scope of the Design Phase

This document covers the design of the entire web application ecosystem, including:

- **System Architecture:** High-level structure (Monolithic MVC).
- **Detailed Component Design:** Class definitions and interaction flows (Sequence Diagrams).
- **Data Design:** Relational database schema and entity relationships.
- **User Interface Design:** Wireframes for critical user journeys.

1.3 Intended Audience

- **Development Team:** To understand the MVC structure, class responsibilities, and database models required for implementation.
- **Testing Team:** To design integration test cases based on the defined sequence flows.
- **Project Supervisors:** To verify that the design aligns with the SRS and course requirements.

1.4 Overview of the Contents

Section 2 provides a high-level system overview. Section 3 details the architectural style and technology stack. Section 4 serves as the core of the document, detailing the MVC pattern application, UML diagrams (Class and Sequence), UI wireframes, and the database schema. Section 5 concludes the design phase.

2. System Overview

2.1 Brief Description of the System

ZC Reclaim is a web-based platform designed to streamline the reporting and recovery of lost items on the Zewail City campus. It replaces fragmented manual processes (emails, social media posts) with a centralized database. Key features include user authentication, item reporting (Lost/Found), advanced search with filtering, and a secure claiming process for found items.

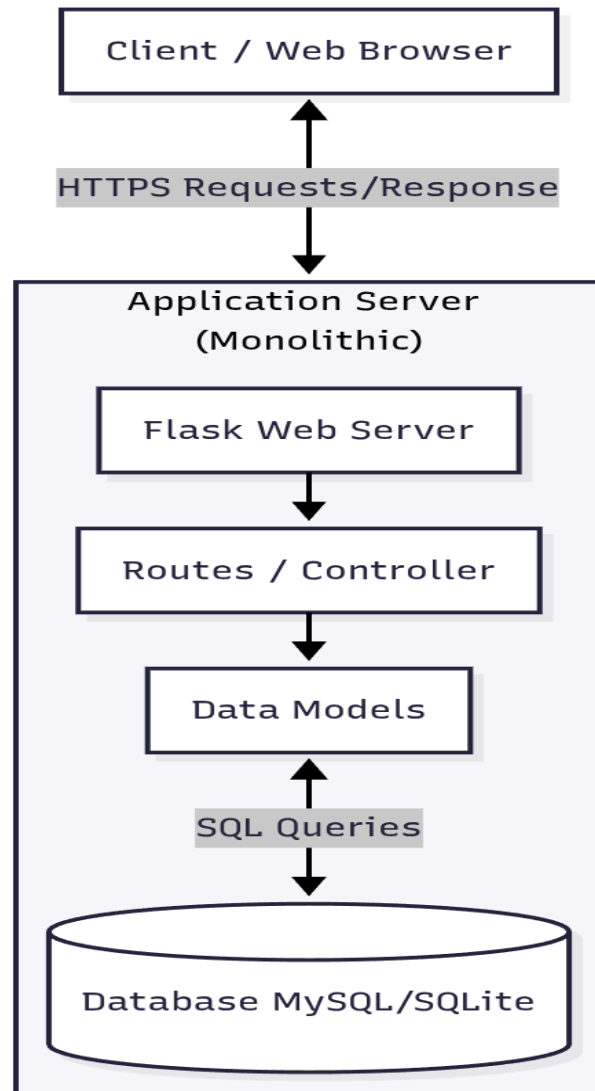
2.2 Key Design Goals and Constraints

- **Performance:** The system is designed to load core pages (Dashboard, Search) in under 3 seconds to meet NFR-1.
- **Data Integrity:** Strict relational constraints are applied to ensure items cannot be claimed by multiple users simultaneously.
- **Security:** Role-Based Access Control (RBAC) is enforced at the Controller level to protect Admin functions.
- **Constraint:** The system must be implemented using **Python Flask** and **HTML** (Server-Side Rendering) without separate frontend frameworks like React, following the **MVC** design pattern.

3. Architectural Design

3.1 System Architecture Diagram

The system follows a traditional web server architecture where the client (Browser) communicates via HTTPS with the Flask Application Server, which interacts with a Relational Database.



3.2 Discussion of Architectural Style and Components

We utilized the **Monolithic Architecture** style.

- **Rationale:** Given the project scale and the team size (2 members), a monolith reduces the overhead of network latency and deployment complexity associated with microservices. It allows for rapid prototyping and strictly adheres to the Flask framework's standard patterns.
- **Components:**

- **Client:** Renders HTML/CSS sent by the server.
- **Flask App:** Handles routing, business logic, and session management.
- **Database:** Stores persistent data for Users, Items, and Claims.

3.3 Technology Stack and Tools

- **Programming Language:** Python
- **Web Framework:** Flask
- **Frontend:** HTML, CSS
- **Database:** SQLite / MySQL
- **ORM:** SQLAlchemy
- **Version Control:** Git & GitHub
- **Design Tools:** Figma(only for the mockup)

4. Detailed Design

4.1 Model-View-Controller (MVC) Design Pattern

4.1.1 Description of MVC Pattern

The MVC pattern separates the application into three logical components to decouple the user interface from the data and business logic.

4.1.2 Mapping of Project Components to MVC

- **Models (/models):** Python classes inheriting from `db.Model` (e.g., `User`, `Item`). These represent the database tables.
- **Views (/templates):** HTML files containing presentation logic (e.g., `report_item.html`, `dashboard.html`).
- **Controllers (/routes):** Python functions decorated with `@app.route`. These handle incoming HTTP requests, process data via Models, and return the appropriate

View.

4.1.3 Responsibilities

- **Model:** Handles data validation (e.g., email format), database storage, and retrieval.
- **View:** Displays data to the user. It is passive and only renders what the Controller provides.
- **Controller:** The "glue" of the system. It interprets user input, invokes business logic in the Model, and selects the View for response.

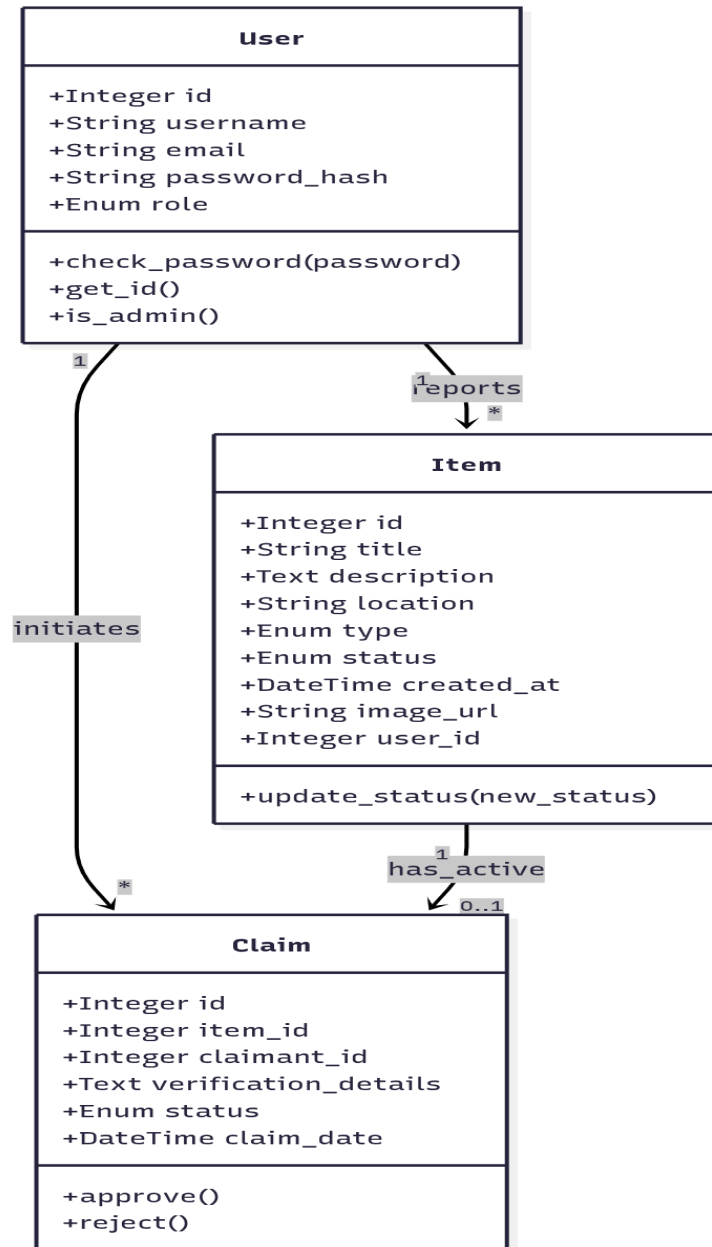
4.1.4 Interaction Flow

1. **User Action:** User clicks "Submit" on the Report Item form.
2. **Controller:** The `report_item()` route receives the POST request.
3. **Model:** The Controller creates a new Item object and calls `db.session.add()`.
4. **View:** Upon success, the Controller redirects to the `item_detail.html` view to show the created post.

4.2 UML Diagrams

4.2.2 Detailed Class Diagram

This diagram defines the static structure of the system classes and their relationships.



4.2.3 Sequence Diagrams

These diagrams illustrate the dynamic behavior of the system for key use cases.

Figure 1: User Login

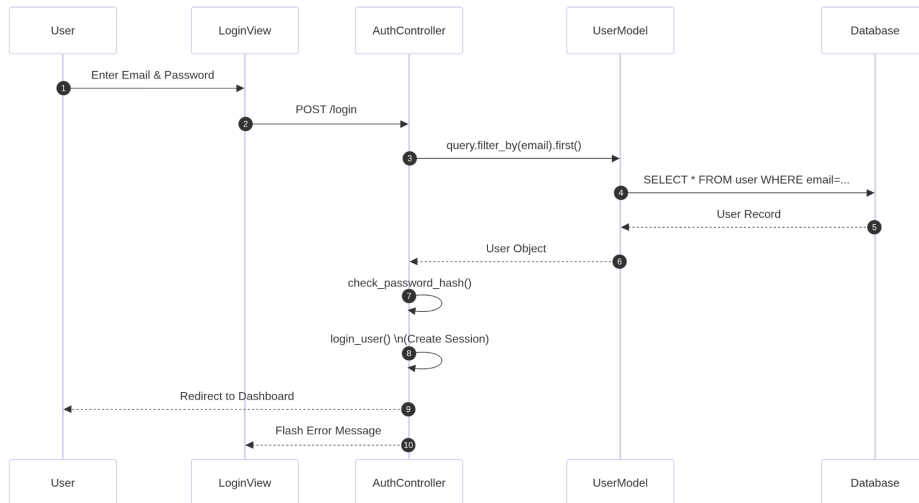


Figure 2: Report Lost Item

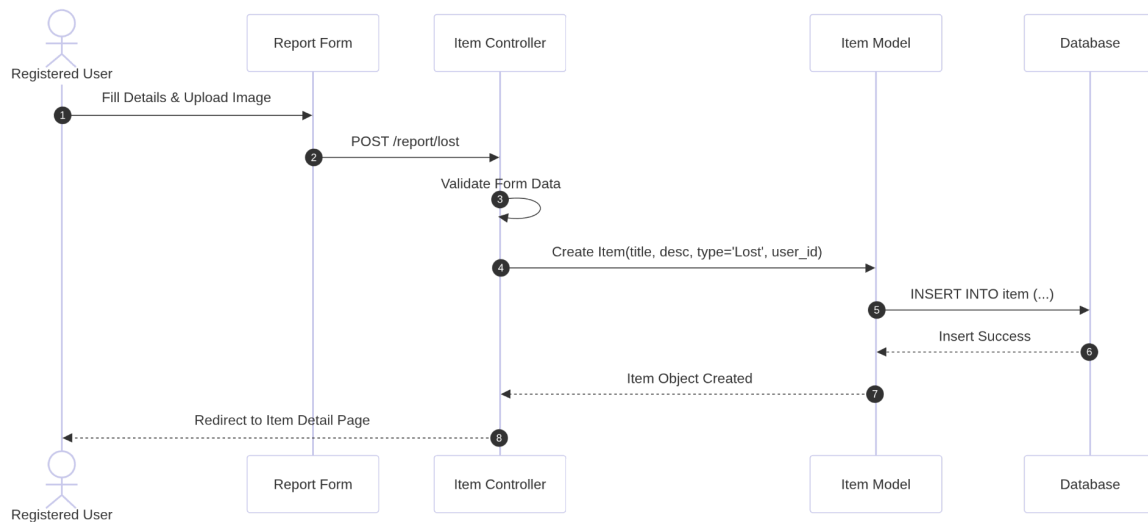


Figure 3: Search for Items

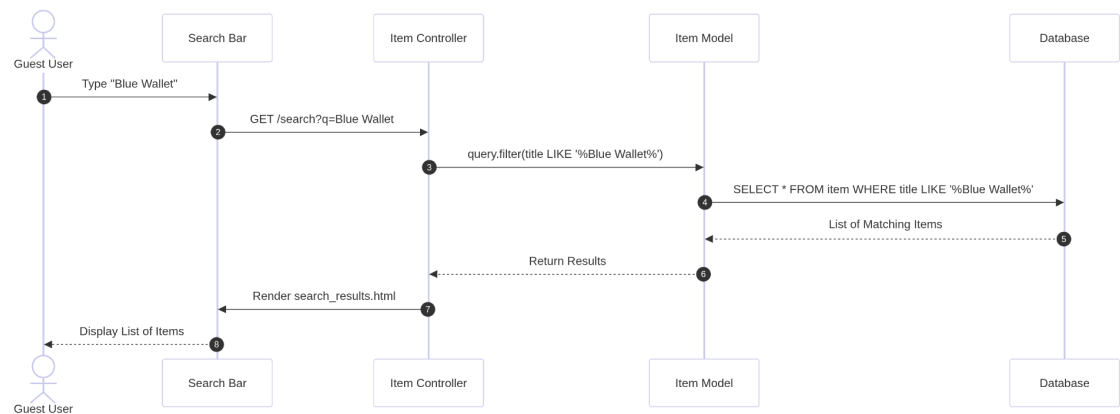


Figure 4: Submit a Claim

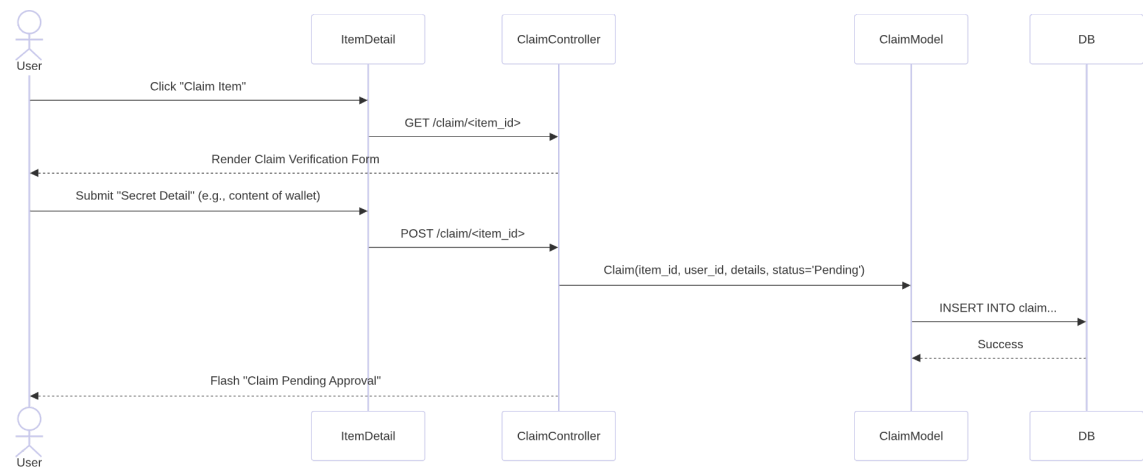
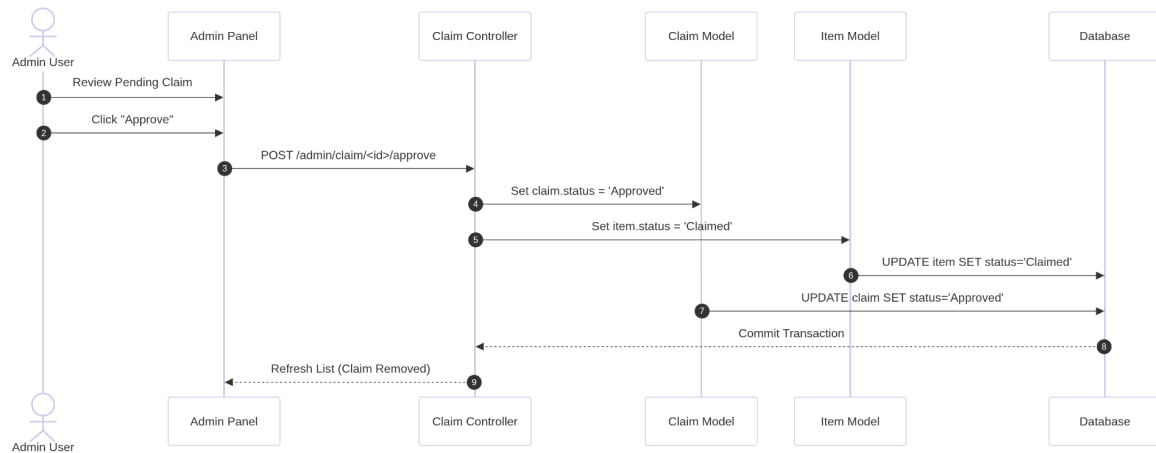


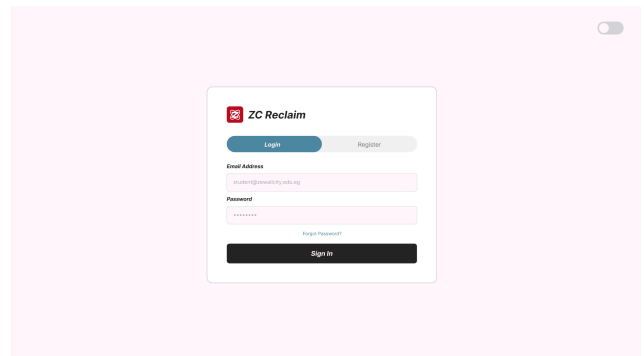
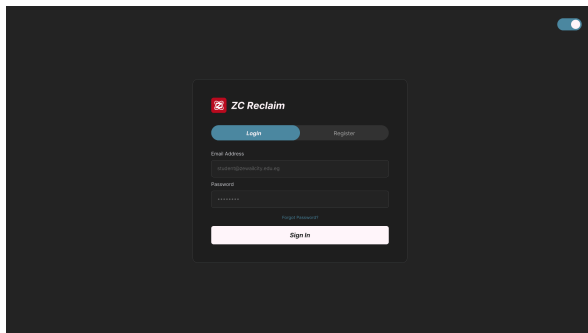
Figure 5: Admin Approves Claim



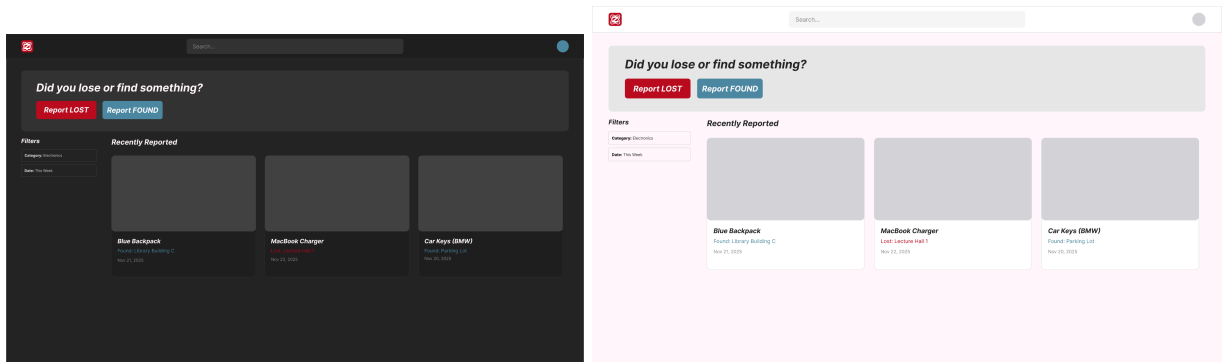
4.3 UI/UX Design

4.3.1 Wireframes / Mockups

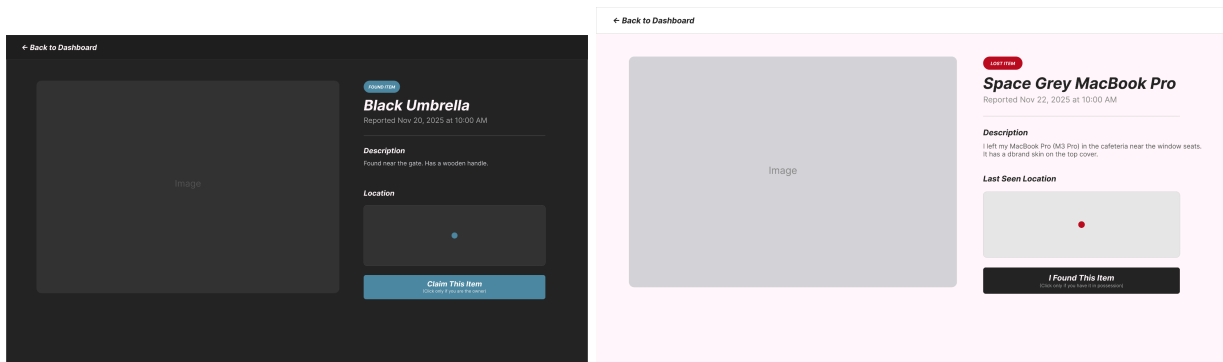
1. Login / Registration Page



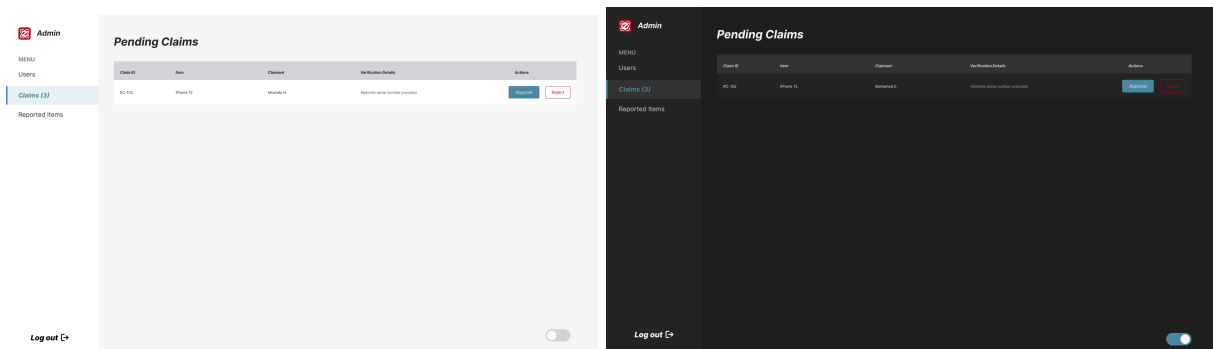
2. Main Dashboard (Home)



3. Item Detail View



4. Admin Dashboard



4.4 Data Design

4.4.1 Database Schema

We will use a Relational Database Management System (RDBMS).

Table: User

Field	Type	Description
id	Int (Primary Key)	Unique identifier
email	String (Unique)	Zewail City email
password_hash	String	Bcrypt hash
role	Enum('Admin', 'Staff', 'User')	User privilege level

Table: Item

Field	Type	Description
id	Int (Primary Key)	Unique identifier
user_id	Int (Foreign Key → User.id)	Creator of the item post
title	String	Short name
type	Enum('Lost', 'Found')	Type of post
status	Enum('Active', 'Claimed', 'Returned')	Item state
description	Text	Detailed information

Table: Claim

Field	Type	Description
id	Int (PK)	Unique identifier
item_id	Int (Foreign Key → Item.id)	Claimed item
claimant_id	Int (Foreign Key → User.id)	User making the claim
verification_details	Text	Proof of ownership
status	Enum('Pending','Approved','Rejected')	Claim progress

4.4.3 Data Dictionary

- **Items.status:** Controls visibility. 'Active' items are visible in search. 'Returned' items are archived.
- **Claims.verification_details:** Confidential field visible only to Admins/Staff for verifying the claimant's ownership (e.g., "The phone has a cat sticker on the back").

5. Conclusion

5.1 Summary of Design Phase

The design phase has successfully established the architectural and logical foundation for **ZC Reclaim**. By adopting the **MVC pattern** within a Monolithic Flask architecture, we ensure the system is modular, testable, and scalable within the scope of the CSAI 203 course. The comprehensive data models and sequence diagrams provide a clear roadmap for Phase 4, allowing the team to proceed directly to the implementation of the Core Functionality Prototype.